

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroler

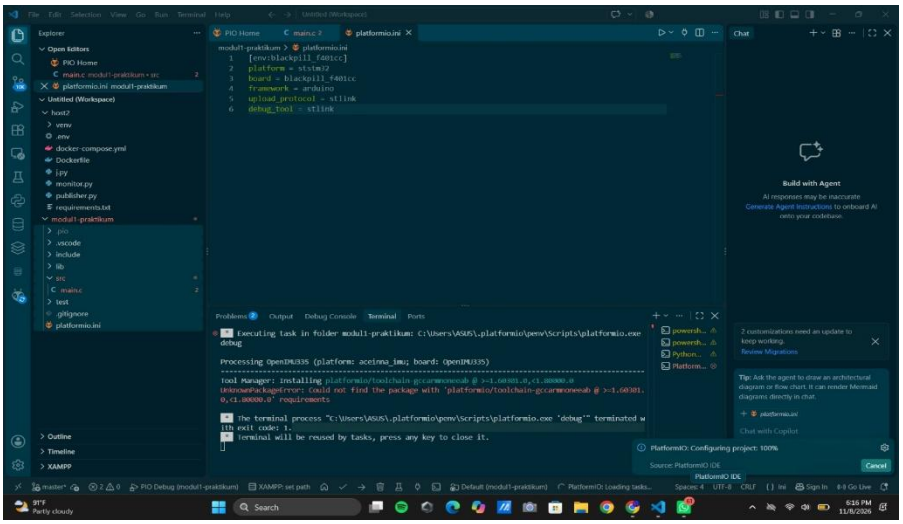
Elmira Araminta Tansy - 5024241099

2026

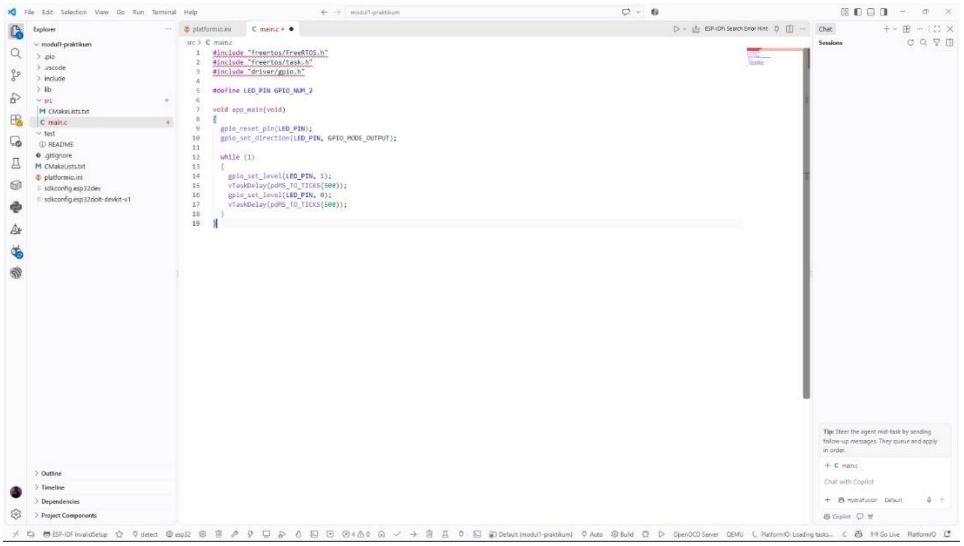
1 Langkah-Langkah Percobaan

Percobaan 1: Blink STM32 Blackpill (Framework Arduino)

1. Pastikan project Modul 1 sudah dibuat, lalu isi platformio.ini dengan:

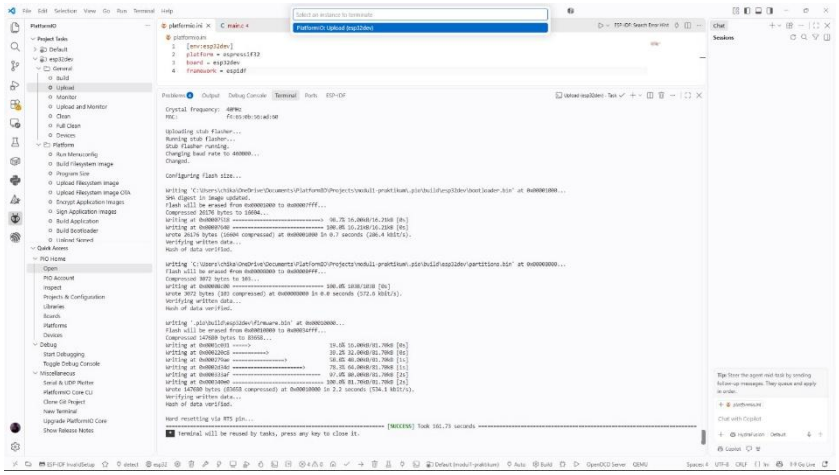


2. Hubungkan ST-Link ke Blackpill dengan pasangan pin SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3, lalu colokkan ST-Link ke komputer lewat USB. Pastikan ST-Link memakai tegangan 3,3 V. Setelah itu, tulis kode Blink pada src/main.cpp, lalu klik Build dan Upload. Hasilnya, LED onboard pada PC13 menyala 0,5 detik dan mati 0,5 detik secara berulang.

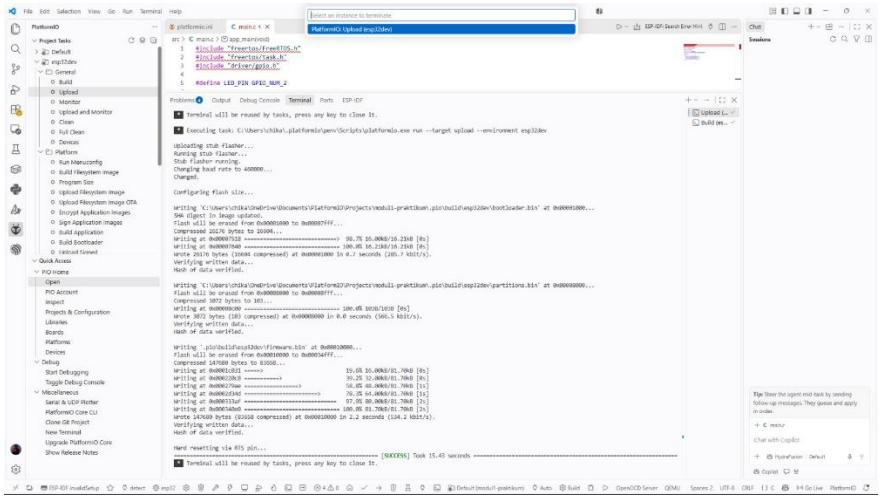


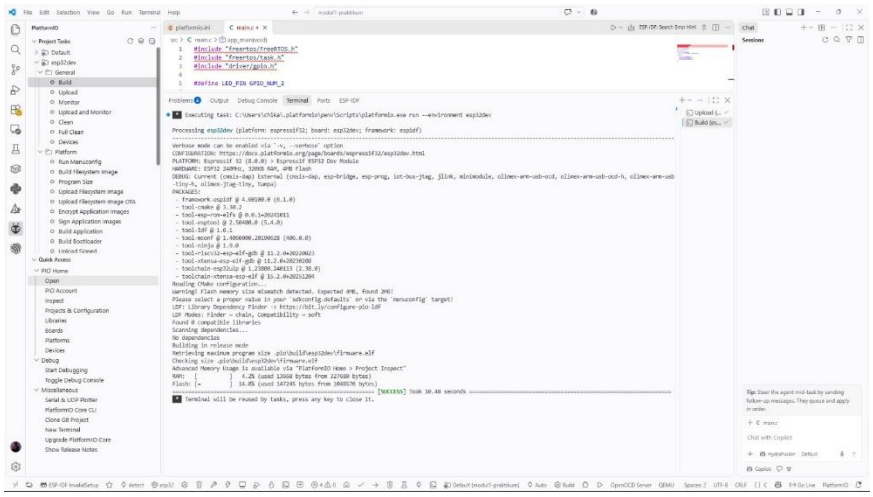
Percobaan 2: Blink ESP32 (Framework ESP-IDF)

Ganti isi platformio.ini dengan konfigurasi berikut dan tunggu proses refresh selesai.



Colokkan ESP32 ke komputer lewat USB dan pastikan port-nya dikenali. Hapus file src/main.cpp, lalu buat file baru src/main.c dan tulis kode berikut. Klik Build dan Upload, lalu amati LED onboard ESP32 yang berkedip. Terakhir, analisis perbedaan memprogram dengan ESP-IDF dan Arduino.



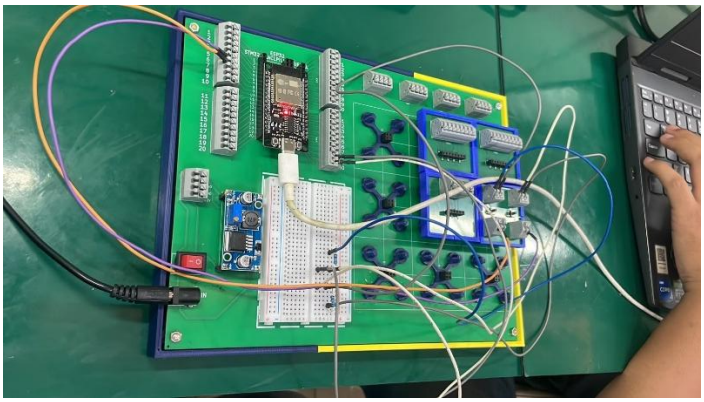


Percobaan 3: Pull-up dan Pull-down (Eksternal vs Internal)

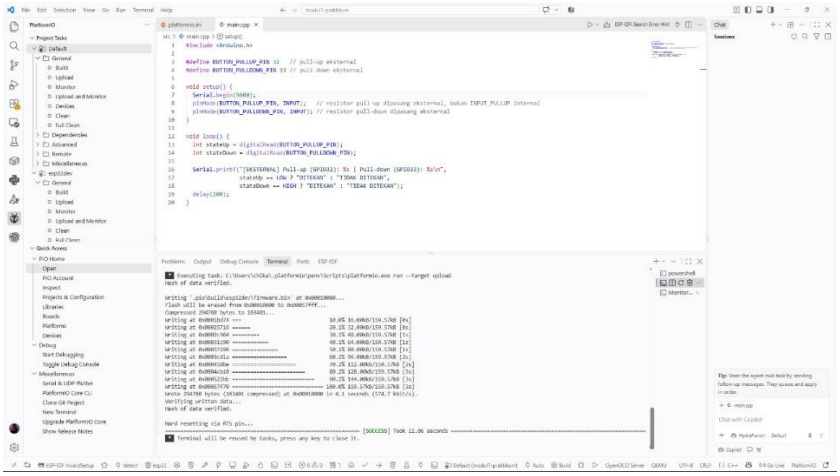
A. Konfigurasi Eksternal

1. Mengganti konfigurasi project pada `platformio.ini` agar menggunakan board ESP32 DevKit dengan framework Arduino.

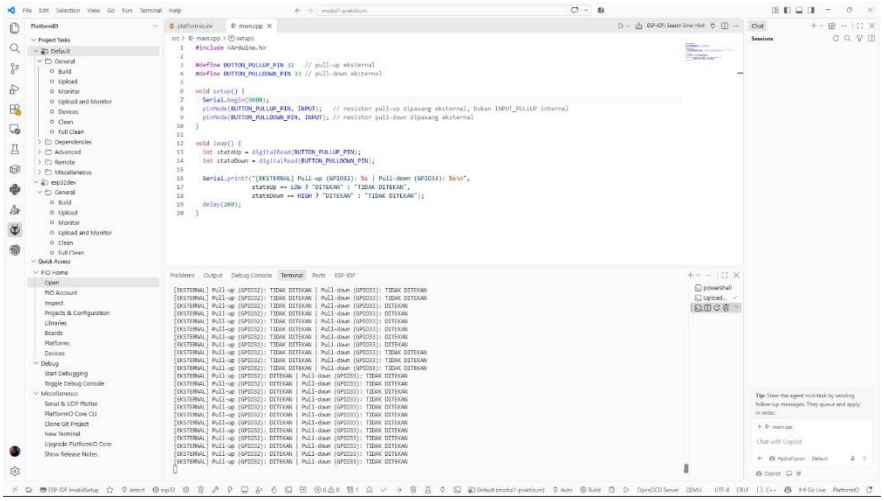
2. Memasang dua pushbutton pada breadboard. Pushbutton pertama dipasang pada GPIO 32 dengan resistor pull-up 10 kΩ, sedangkan pushbutton kedua dipasang pada GPIO 33 dengan resistor pull-down 10 kΩ. *Gambar 6: Rangkaian pushbutton dengan pull-up dan pull-down eksternal sesuai Modul 1 Percobaan 3.*



3. Menyambungkan rangkaian sesuai skema pada modul. Pada pull-up, pin GPIO dan satu kaki tombol dihubungkan ke 3,3 V melalui resistor, sedangkan kaki tombol lainnya dihubungkan ke GND. Pada pull-down, pin GPIO dan satu kaki tombol dihubungkan ke GND melalui resistor, sedangkan kaki tombol lainnya dihubungkan ke 3,3 V.
4. Menulis program untuk membaca kondisi kedua tombol menggunakan fungsi `digitalRead()`.
5. Melakukan Build dan Upload, lalu membuka Serial Monitor dengan baud rate 115200.

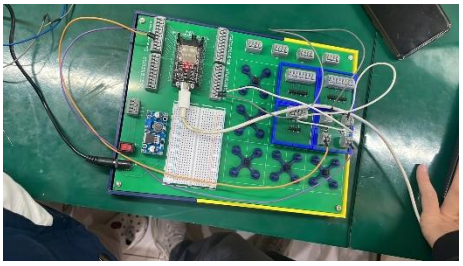


6. Menekan dan melepas kedua tombol secara bergantian untuk mengamati perubahan kondisi logika yang tampil pada Serial Monitor.

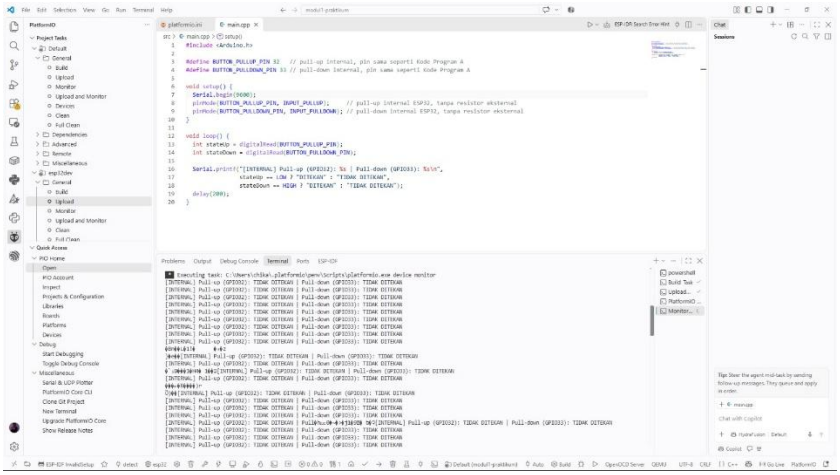


B. Konfigurasi Internal

1. Melepas kedua resistor eksternal dari rangkaian pushbutton, sementara posisi tombol dan koneksi ke GPIO tetap dipertahankan. *Gambar 8: Rangkaian pushbutton tanpa resistor eksternal sesuai Modul 1 Percobaan 3.*

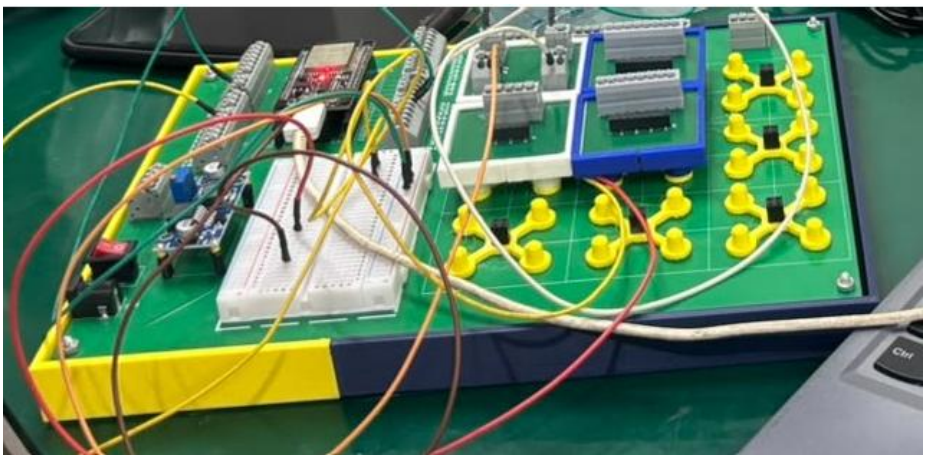


2. Mengubah konfigurasi pin pada program menjadi `INPUT_PULLUP` untuk GPIO 32 dan `INPUT_PULLDOWN` untuk GPIO 33.
3. Melakukan Build dan Upload ulang program ke ESP32.
4. Membuka Serial Monitor, lalu menguji kedua tombol dengan menekan dan melepasnya secara bergantian.
5. Membandingkan hasil pembacaan pada konfigurasi internal dengan hasil pada konfigurasi eksternal.

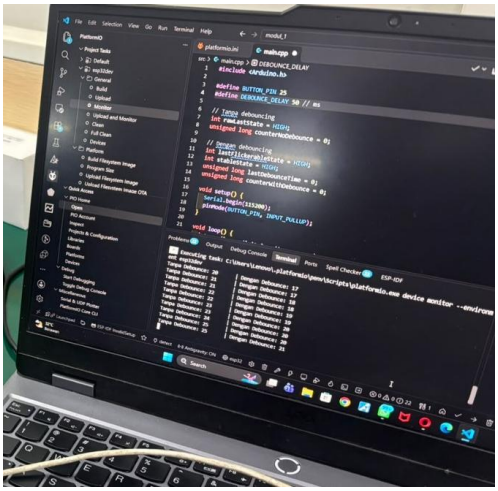


Percobaan 4: Debouncing pada Input Tombol GPIO

1. Memasang satu pushbutton pada GPIO 25 ESP32 dengan konfigurasi pull-up internal. *Gambar 10: Rangkaian pushbutton pada GPIO 25 dengan pull-up internal sesuai Modul 1 Percobaan 4.*

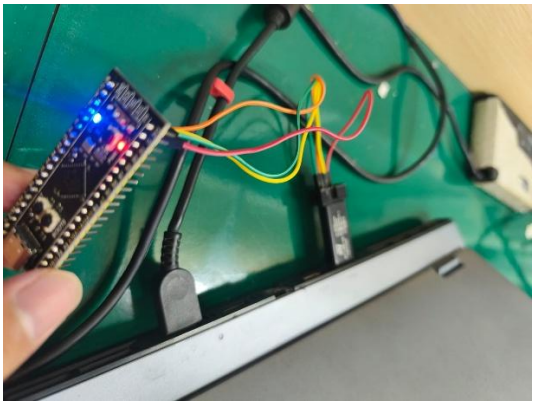


2. Menulis program yang membandingkan dua metode pembacaan tombol, yaitu tanpa debouncing dan dengan debouncing.
3. Melakukan Build dan Upload, lalu membuka Serial Monitor untuk mengamati nilai kedua penghitung.
4. Menekan tombol satu kali, lalu membandingkan perubahan nilai counterNoDebounce dengan counterWithDebounce.
5. Mengubah nilai DEBOUNCE_DELAY menjadi 10 ms, kemudian melakukan Build dan Upload ulang.
6. Mengulangi pengujian dengan mengubah nilai DEBOUNCE_DELAY menjadi 200 ms.
7. Membandingkan hasil pengujian pada waktu debouncing 10 ms, 50 ms, dan 200 ms untuk mengetahui pengaruh interval waktu terhadap pembacaan tombol.

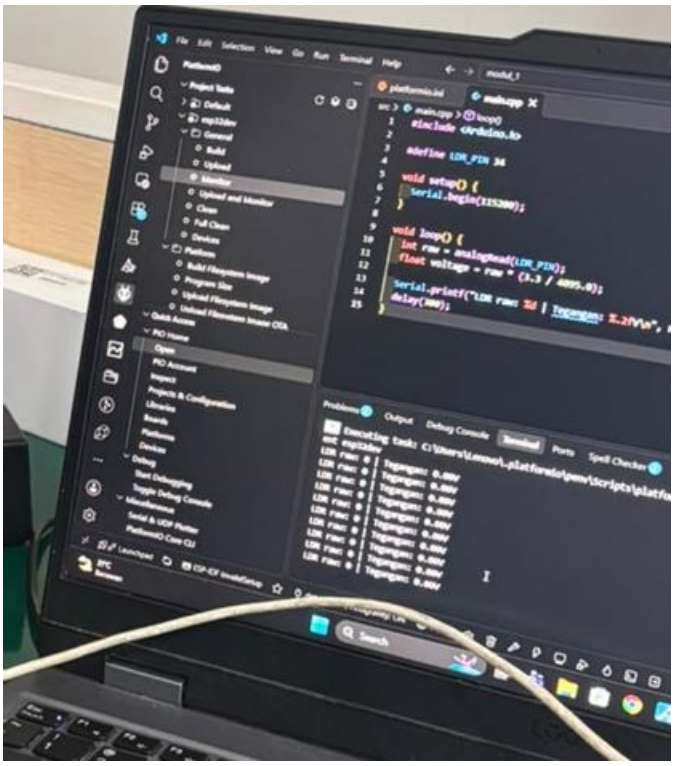


Percobaan 5: Pembacaan ADC dengan Sensor LDR

1. Menyusun rangkaian pembagi tegangan pada breadboard menggunakan sensor LDR dan resistor 10 kΩ.
2. Menghubungkan satu kaki LDR ke sumber tegangan 3,3 V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan ke GND melalui resistor 10 kΩ.
3. Mengganti konfigurasi project pada platformio.ini agar menggunakan board ESP32 DevKit dengan framework Arduino.
4. Menulis program untuk membaca nilai ADC dari GPIO 34 menggunakan fungsi analogRead().
5. Menambahkan perhitungan untuk mengonversi nilai ADC menjadi perkiraan tegangan berdasarkan rentang pembacaan 0-4095.
6. Melakukan Build dan Upload, lalu membuka Serial Monitor dengan baud rate 115200.
7. Mengamati perubahan nilai ADC dan tegangan saat permukaan LDR ditutup dengan tangan, kemudian disinari dengan lampu. *Gambar 12: Perubahan nilai ADC saat LDR disinari lampu.*



8. Membandingkan perubahan nilai pembacaan ADC berdasarkan perbedaan intensitas cahaya yang diterima LDR.



2 Analisis Hasil Percobaan

Pada percobaan pertama dan kedua, dilakukan pengujian LED Blink menggunakan STM32 Blackpill dengan framework Arduino (pin PC13) dan ESP32 DevKit dengan framework ESP-IDF (GPIO 2). Kedua program mengatur pin sebagai output sehingga LED menyala dan mati bergantian dengan jeda 500 ms. Pada STM32, LED bawaan bersifat aktif LOW sehingga menyala ketika pin diberi LOW, sedangkan pada ESP32 pengaturan waktu memakai vTaskDelay() dari FreeRTOS. Hasil ini menunjukkan bahwa prinsip dasar pengendalian GPIO output sama pada kedua board, dan perbedaannya hanya pada framework serta fungsi yang digunakan. Pada percobaan pertama sempat muncul kendala, yaitu PlatformIO bermasalah di laptop awal sehingga proses upload gagal. Masalah ini teratasi setelah praktikan beralih ke laptop lain.

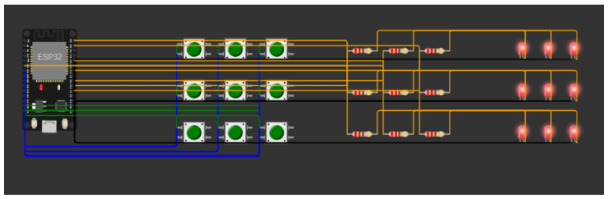
Pada percobaan ketiga dan keempat, dilakukan pengujian input tombol pada ESP32. Pada percobaan ketiga, konfigurasi pull-up (GPIO 32) dan pull-down (GPIO 33) diuji dengan resistor eksternal dan resistor internal (INPUT_PULLUP dan INPUT_PULLDOWN). Hasil di Serial Monitor sesuai teori, yaitu pull-up menjaga kondisi awal pin pada HIGH dan pull-down menjaga kondisi awal pin pada LOW. Resistor internal membuat rangkaian lebih sederhana, sedangkan resistor eksternal memberi kebebasan lebih besar dalam menentukan konfigurasi.

Pada percobaan keempat, pembacaan tombol di GPIO 25 dibandingkan antara kondisi tanpa dan dengan debouncing, dengan waktu debounce 10 ms, 50 ms, dan 200 ms. Jumlah pembacaan dengan debouncing lebih sedikit karena sinyal tidak stabil akibat bouncing berhasil disaring, sehingga satu kali penekanan tidak terbaca berulang. Waktu debounce yang terlalu singkat belum tentu menyaring bouncing dengan baik, sedangkan waktu yang terlalu lama membuat respons tombol menjadi lambat.

Pada percobaan kelima, sensor LDR dihubungkan ke GPIO 34 dan dibaca dengan analogRead(), lalu nilainya dikonversi menjadi tegangan. Nilai ADC berubah pada rentang 0 hingga 4095 dengan tegangan sekitar 0 hingga 3,3 V, sesuai resolusi ADC 12-bit pada ESP32. Perubahan ini terjadi karena hambatan LDR berubah mengikuti intensitas cahaya, sehingga tegangan pada rangkaian pembagi tegangan ikut berubah, dengan arah perubahan yang bergantung pada susunan LDR dan resistor. Percobaan ini menunjukkan bahwa mikrokontroler dapat membaca sinyal analog dari sensor untuk mengetahui kondisi lingkungan.

Secara keseluruhan, kendala PlatformIO pada laptop awal menunjukkan bahwa kesiapan perangkat dan lingkungan pengembangan memengaruhi kelancaran praktikum.

3 Hasil Tugas Modul



```
Matrix Pushbutton 3x3 -> Matrix LED 3x3
Tombol (1,1) -> LED ON
Tombol (1,1) -> LED OFF
Tombol (1,2) -> LED ON
Tombol (1,2) -> LED OFF
Tombol (1,3) -> LED ON
Tombol (1,3) -> LED OFF
Tombol (2,1) -> LED ON
Tombol (2,1) -> LED OFF
Tombol (2,2) -> LED ON
Tombol (2,2) -> LED OFF
Tombol (2,3) -> LED ON
Tombol (2,3) -> LED OFF
Tombol (3,1) -> LED ON
Tombol (3,1) -> LED OFF
Tombol (3,2) -> LED ON
Tombol (3,2) -> LED OFF
Tombol (3,3) -> LED ON
Tombol (3,3) -> LED OFF
```

```
#include <Arduino.h>

2

3 const int rowPins[3] = {13, 14, 27};

4 const int colPins[3] = {26, 25, 33};

5 const int ledPins[3][3] = {

6 {16, 17, 18},

7 {19, 21, 22},

8 {23, 32, 4}

9 };

10

11 const unsigned long debounceDelay = 50;

12

13 bool lastReading[3][3];

14 bool stableState[3][3];

15 unsigned long lastChange[3][3];

16 bool ledOn[3][3];
```

```
17
18 void scanMatrix(bool raw[3][3]) {
19 for (int r = 0; r < 3; r++) {
20 for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
21 digitalWrite(rowPins[r], LOW);
22 delayMicroseconds(10);
23 for (int c = 0; c < 3; c++) {
24 raw[r][c] = (digitalRead(colPins[c]) == LOW);
25 }
26 }
27 for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
28 }
29
30 void setup() {
31 Serial.begin(115200);
32
33 for (int i = 0; i < 3; i++) {
34 pinMode(rowPins[i], OUTPUT);
35 digitalWrite(rowPins[i], HIGH);
36 pinMode(colPins[i], INPUT_PULLUP);
37 }
38
39 for (int r = 0; r < 3; r++) {
40 for (int c = 0; c < 3; c++) {
41 pinMode(ledPins[r][c], OUTPUT);
42 digitalWrite(ledPins[r][c], LOW);
43 lastReading[r][c] = false;
44 stableState[r][c] = false;
45 lastChange[r][c] = 0;
46 ledOn[r][c] = false;
47 }
48 }
49
50 Serial.println("Matrix Pushbutton 3x3-> Matrix LED 3x3");
51 }
```

```
52
53 void loop() {
9
54 bool raw[3][3];
55 scanMatrix(raw);
56 unsigned long now = millis();
57
58 for (int r = 0; r < 3; r++) {
59 for (int c = 0; c < 3; c++) {
60 if (raw[r][c] != lastReading[r][c]) {
61 lastChange[r][c] = now;
62 lastReading[r][c] = raw[r][c];
63 }
64
65 if ((now- lastChange[r][c]) > debounceDelay && raw[r][c] != stableState[r][c
]) {
66 stableState[r][c] = raw[r][c];
67
68 if (stableState[r][c]) {
69 ledOn[r][c] = !ledOn[r][c];
70 digitalWrite(ledPins[r][c], ledOn[r][c] ? HIGH : LOW);
71 Serial.printf("Tombol (%d,%d)-> LED %s\n", r + 1, c + 1, ledOn[r][c] ? "
ON" : "OFF");
72 }
73 }
74 }
75 }
76 }
77
```

Jawaban Pertanyaan Analisis

1. Bouncing terjadi di dalam tombol itu sendiri, bukan karena banyaknya tombol yang ditekan. Seperti pada teori C.5, kontak logam pada tombol mekanis memantul beberapa kali dalam waktu sangat singkat saat ditekan atau dilepas, sehingga sinyal berubah HIGH dan LOW berulang kali sebelum stabil. Karena ESP32 membaca jauh lebih cepat daripada durasi bouncing, satu kali penekanan dapat terbaca sebagai beberapa kali penekanan. Pada program ini, LED berubah (toggle) setiap penekanan, sehingga tanpa debouncing LED dapat berganti ON/OFF berkali-kali dan

hasil akhirnya menjadi salah. Dengan debouncing, LED baru berubah setelah pembacaan tombol stabil selama 50 ms.

2. Kolom menggunakan pull-up internal (INPUT_PULLUP), sehingga saat tombol tidak ditekan, pin bernilai HIGH. Saat tombol ditekan, pin terhubung ke baris yang bernilai LOW sehingga pin bernilai LOW. Pin tidak boleh dibiarkan floating karena mudah terpengaruh noise. Akibatnya nilainya dapat berubah acak antara HIGH dan LOW, dan LED dapat berubah sendiri tanpa ada tombol yang ditekan.

3. Digital input hanya membandingkan tegangan dengan batas ambang, sehingga hanya ada dua kondisi, yaitu HIGH atau LOW (1 bit). ADC 12-bit pada ESP32 mengubah tegangan 0-3,3 V menjadi 4096 tingkat nilai (0-4095). Jika tombol diganti dengan LDR, tegangan berubah secara kontinu sesuai intensitas cahaya, sehingga ADC dapat membedakan banyak tingkat terang-gelap, bukan hanya terang atau gelap.

4 Kesimpulan

Praktikum ini berjalan dengan baik dan seluruh hasilnya sesuai dengan teori. Praktikan mampu memprogram STM32 Blackpill dan ESP32 menggunakan PlatformIO, mengatur GPIO sebagai output maupun input, serta memahami fungsi pull-up dan pull-down, debouncing, dan pembacaan ADC pada sensor LDR. Pada tugas modul, matrix pushbutton 3x3 berhasil mengendalikan matrix LED 3x3 pada simulasi. Debouncing diterapkan sehingga LED hanya berubah setelah pembacaan tombol stabil. Kendala PlatformIO pada laptop awal dapat diatasi dengan beralih ke laptop lain

5 Lampiran

5.1 Dokumentasi saat praktikum

